

ES6와 배열 메서드



let, const

```
let apple = "사과";  
const banana = "바나나";
```

`var`이 아닌 다른 키워드로도 변수 선언이 가능한데,
`let`, `const`를 사용하면 된다.

```
let fruit = "오렌지";  
fruit = "포도";
```

```
let elice;  
elice = "hellobit";
```

‘let’ 키워드는 **값이 변경 가능**한 변수를 선언하는 키워드이다.

따라서 선언과 초기화를 동시에 하지 않아도 된다.

```
const fruit = "오렌지";  
  
fruit = "오렌지"; // error  
const elice;      // error
```

‘const’ 키워드는 **값의 변경이 불가능**한 변수를 선언하는 키워드이다. (=상수)

따라서 **선언과 초기화를 무조건 동시에** 해야 한다.



var vs let, const

```
let apple = "사과";  
const banana = "바나나";
```

```
var fruit = "사과";  
var apple = "바나나";
```

이제부터 변수 선언 시에는 **무조건 `let` 혹은 `const`를 사용**하는 것을 추천,
`var`는 사용하지 않는 것을 추천한다.



var vs let, const

```
var fruit = "오렌지";  
var fruit = "포도";
```

```
let fruit = "오렌지";  
let fruit = "포도"; // error  
  
const elice = "hellobit";  
const elice = "caterpillar"; // error
```

`var` 키워드로는 변수 중복 선언이 허용된다.
반면 `let`, `const`는 변수 중복 선언 시 에러가 발생한다.



var vs let, const

```
var fruit = "오렌지";  
var fruit = "포도";
```

```
let fruit = "오렌지";  
let fruit = "포도"; // error  
  
const elice = "hellobit";  
const elice = "caterpillar"; // error
```

`let`, `const`를 사용하면 **의도치 않은 변수 변경을 방지**할 수 있다.



var vs let, const

함수 레벨 스코프 vs 블록 레벨 스코프

변수 Hoisting

이 외에도 두 가지 차이점이 더 존재한다.



스코프 (Scope)

```
var a = 1;

if (a > 0) {
  var b = 2;
}

function func(c) {
  var d = 4;

  console.log("console log in function")
  console.log(a);
  console.log(b);
  console.log(c);
  console.log(d);
}
```

```
func(3);

console.log("console log in global");
console.log(a);
console.log(b);
console.log(c);
console.log(d);
```

스코프란 변수를 참조할 수 있는 유효 범위를 의미한다.



스코프 (Scope)

```
var a = 1;

if (a > 0) {
  var b = 2;
}

function func(c) {
  var d = 4;

  console.log("console log in function")
  console.log(a);
  console.log(b);
  console.log(c);
  console.log(d);
}
```

```
func(3);

console.log("console log in global");
console.log(a);
console.log(b);
console.log(c);
console.log(d);
```



```
'console log in function'
1
2
3
4
'console log in global'
1
2
ReferenceError: c is not defined
```

스코프란 변수를 참조할 수 있는 유효 범위를 의미한다.



전역 스코프 vs 지역 스코프

종류	설명
전역 스코프	가장 바깥의 코드 영역
지역 스코프	함수 및 코드 블록 내부

스코프는 크게 전역 스코프와 지역 스코프로 나뉜다.



전역 스코프 (Global Scope)

```
var a = 1;

if (a > 0) {
  var b = 2;
}

function func(c) {
  var d = 4;

  console.log("console log in function")
  console.log(a);
  console.log(b);
  console.log(c);
  console.log(d);
}

func(3);

console.log("console log in global");
console.log(a);
console.log(b);
console.log(c);
console.log(d);
```

전역 스코프



지역 스코프 (Local Scope)

```
var a = 1;
```

```
if (a > 0) {
```

```
  var b = 2;
```

```
}
```

지역 스코프

```
function func(c) {
```

```
  var d = 4;
```

지역 스코프

```
  console.log("console log in function")
```

```
  console.log(a);
```

```
  console.log(b);
```

```
  console.log(c);
```

```
  console.log(d);
```

```
}
```

```
func(3);
```

```
console.log("console log in global");
```

```
console.log(a);
```

```
console.log(b);
```

```
console.log(c);
```

```
console.log(d);
```




지역 스코프 (Local Scope)

지역 스코프를 어디까지 인정할 것인가?



지역 스코프 (Local Scope)



종류	설명
함수 레벨 스코프	함수 내부만 지역 스코프로 인정
블록 레벨 스코프	함수 내부 및 코드 블록 내부 둘 다 지역 스코프로 인정

지역 스코프를 어디까지 인정할 것인가?



함수 레벨 스코프 - var

```
var a = 1;

if (a > 0) {
  var b = 2;
}

function func(c) {
  var d = 4;

  console.log("console log in function")
  console.log(a);
  console.log(b);
  console.log(c);
  console.log(d);
}

func(3);

console.log("console log in global");
console.log(a);
console.log(b);
console.log(c);
console.log(d);
```



```
'console log in function'
1
2
3
4
'console log in global'
1
2
ReferenceError: c is not defined
```

‘var’ 키워드는 함수 레벨 스코프를 지원하지므로,
if문 내부의 변수 ‘b’는 전역 변수가 된다.



블록 레벨 스코프 – let, const

```
const a = 1;

if (a > 0) {
  const b = 2;
}

function func(c) {
  const d = 4;

  console.log("console log in function");
  console.log(a);
  console.log(b);
  console.log(c);
  console.log(d);
}

func(3);

console.log("console log in global");
console.log(a);
console.log(b);
console.log(c);
console.log(d);
```



```
'console log in function'
1
ReferenceError: b is not defined
```

반면 `let`, `const` 키워드는 블록 레벨 스코프를 지원하므로,
if문 내부의 변수 `b`는 지역 변수가 된다.



스코프 체인

```
const global = 10;

function outerFunc() {
  const global = 20;
  const local1 = 30;

  function innerFunc() {
    const local1 = 40;
    const local2 = 50;

    console.log("inner function");
    console.log(global);
    console.log(local1);
    console.log(local2);
  }

  console.log("outer function");
  console.log(global);
  console.log(local1);

  innerFunc();
}

console.log("global");
console.log(global);

outerFunc();
```




스코프 체인

```
const global = 10;

function outerFunc() {
  const global = 20;
  const local1 = 30;

  function innerFunc() {
    const local1 = 40;
    const local2 = 50;

    console.log("inner function");
    console.log(global);
    console.log(local1);
    console.log(local2);
  }

  console.log("outer function");
  console.log(global);
  console.log(local1);

  innerFunc();
}

console.log("global");
console.log(global);

outerFunc();
```



```
'inner function'
20
40
50
'outer function'
20
30
'global'
10
```




스코프 체인

```
const global = 10;

function outerFunc() {
  const global = 20;
  const local1 = 30;

  function innerFunc() {
    const local1 = 40;
    const local2 = 50;

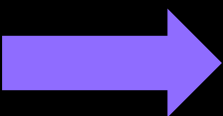
    console.log("inner function");
    console.log(global);
    console.log(local1);
    console.log(local2);
  }

  console.log("outer function");
  console.log(global);
  console.log(local1);

  innerFunc();
}

console.log("global");
console.log(global);

outerFunc();
```



전역 스코프	
global	10
outerFunc	f outerFunc()



스코프 체인

```
const global = 10;

function outerFunc() {
  const global = 20;
  const local1 = 30;

  function innerFunc() {
    const local1 = 40;
    const local2 = 50;

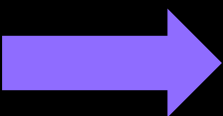
    console.log("inner function");
    console.log(global);
    console.log(local1);
    console.log(local2);
  }

  console.log("outer function");
  console.log(global);
  console.log(local1);

  innerFunc();
}

console.log("global");
console.log(global);

outerFunc();
```



전역 스코프	
global	10
outerFunc	f outerFunc()

outerFunc 지역 스코프	
global	20
local1	30
innerFunc	f innerFunc()



스코프 체인

```
const global = 10;

function outerFunc() {
  const global = 20;
  const local1 = 30;

  function innerFunc() {
    const local1 = 40;
    const local2 = 50;

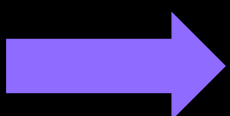
    console.log("inner function");
    console.log(global);
    console.log(local1);
    console.log(local2);
  }

  console.log("outer function");
  console.log(global);
  console.log(local1);

  innerFunc();
}

console.log("global");
console.log(global);

outerFunc();
```



전역 스코프	
global	10
outerFunc	f outerFunc()
global	20
local1	30
innerFunc	f innerFunc()

innerFunc 지역 스코프	
local1	40
local2	50



스코프 체인

```
const global = 10;

function outerFunc() {
  const global = 20;
  const local1 = 30;

  function innerFunc() {
    const local1 = 40;
    const local2 = 50;

    console.log("inner function");
    console.log(global);
    console.log(local1);
    console.log(local2);
  }

  console.log("outer function");
  console.log(global);
  console.log(local1);

  innerFunc();
}

console.log("global");
console.log(global);

outerFunc();
```



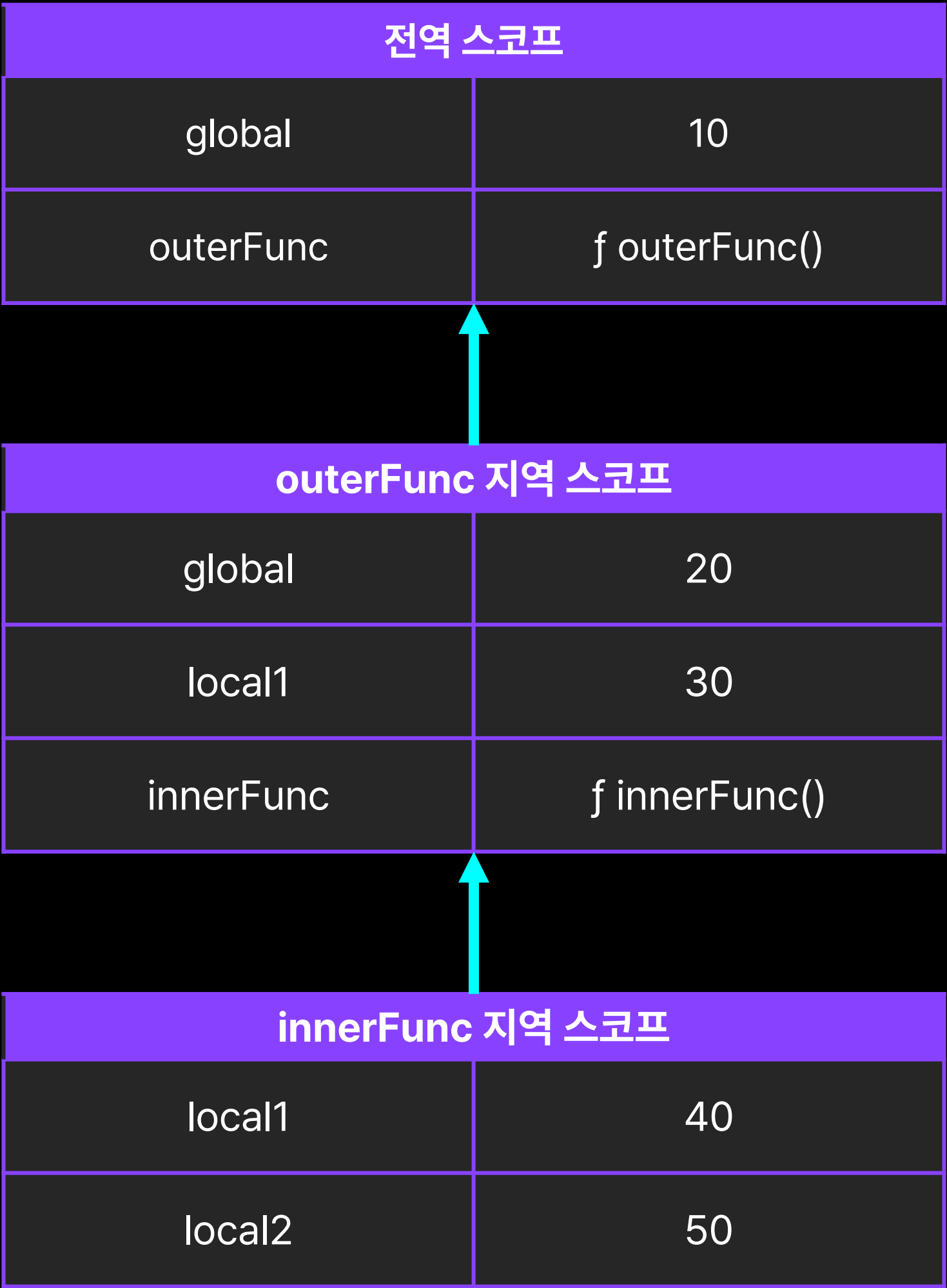
전역 스코프	
global	10
outerFunc	f outerFunc()

outerFunc 지역 스코프	
global	20
local1	30
innerFunc	f innerFunc()

innerFunc 지역 스코프	
local1	40
local2	50

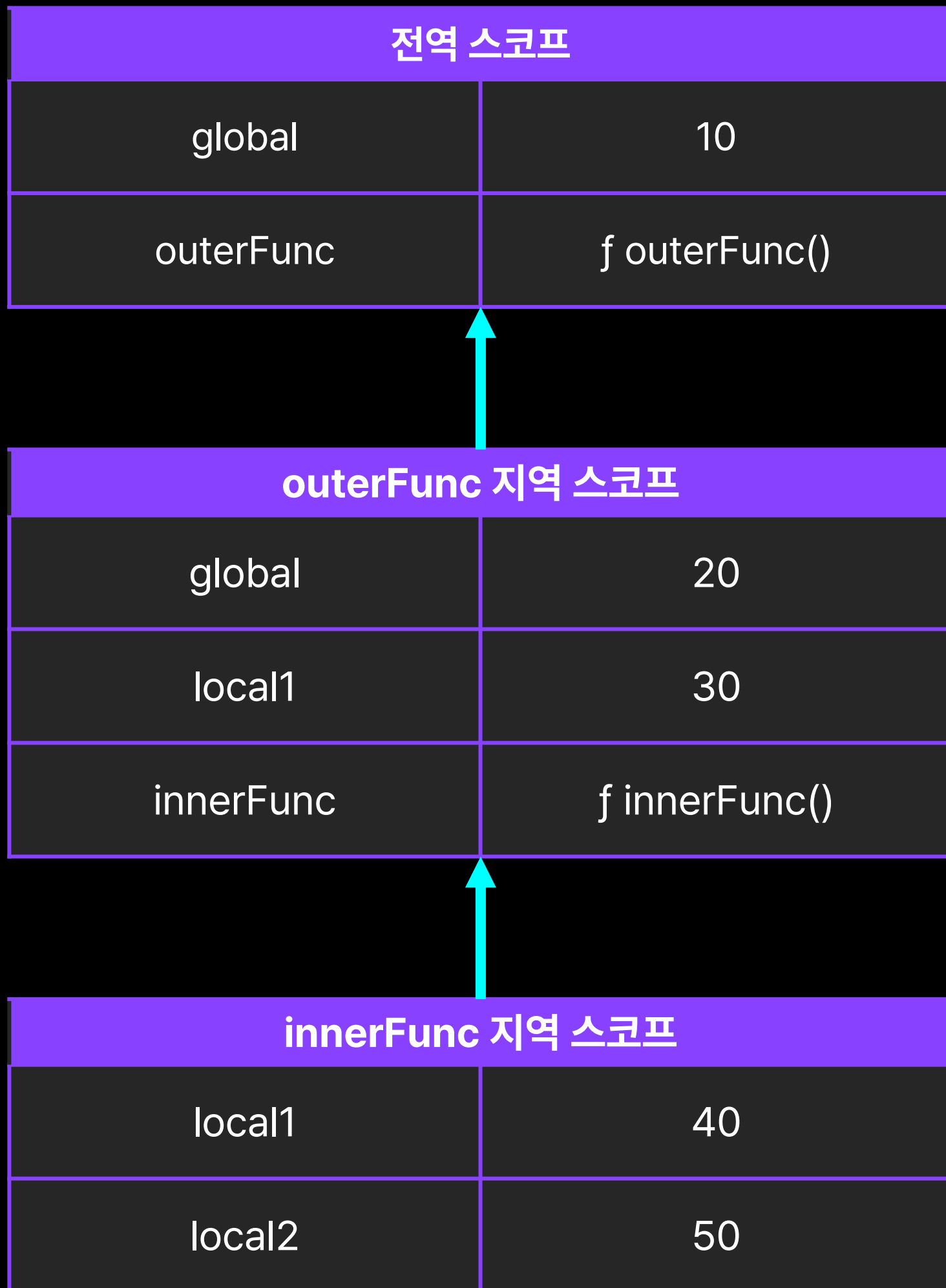


스코프 체인





스코프 체인



- 스코프 체인이란 모든 스코프가 **계층적 구조로 연결된** 것을 의미한다.
- 모든 지역 스코프의 최상위 스코프는 전역 스코프이다.
- 변수 및 함수를 참조할 때 해당 스코프에서 시작해 **상위 스코프 방향으로 순차적으로 이동**하면서 변수 및 함수를 검색한다.

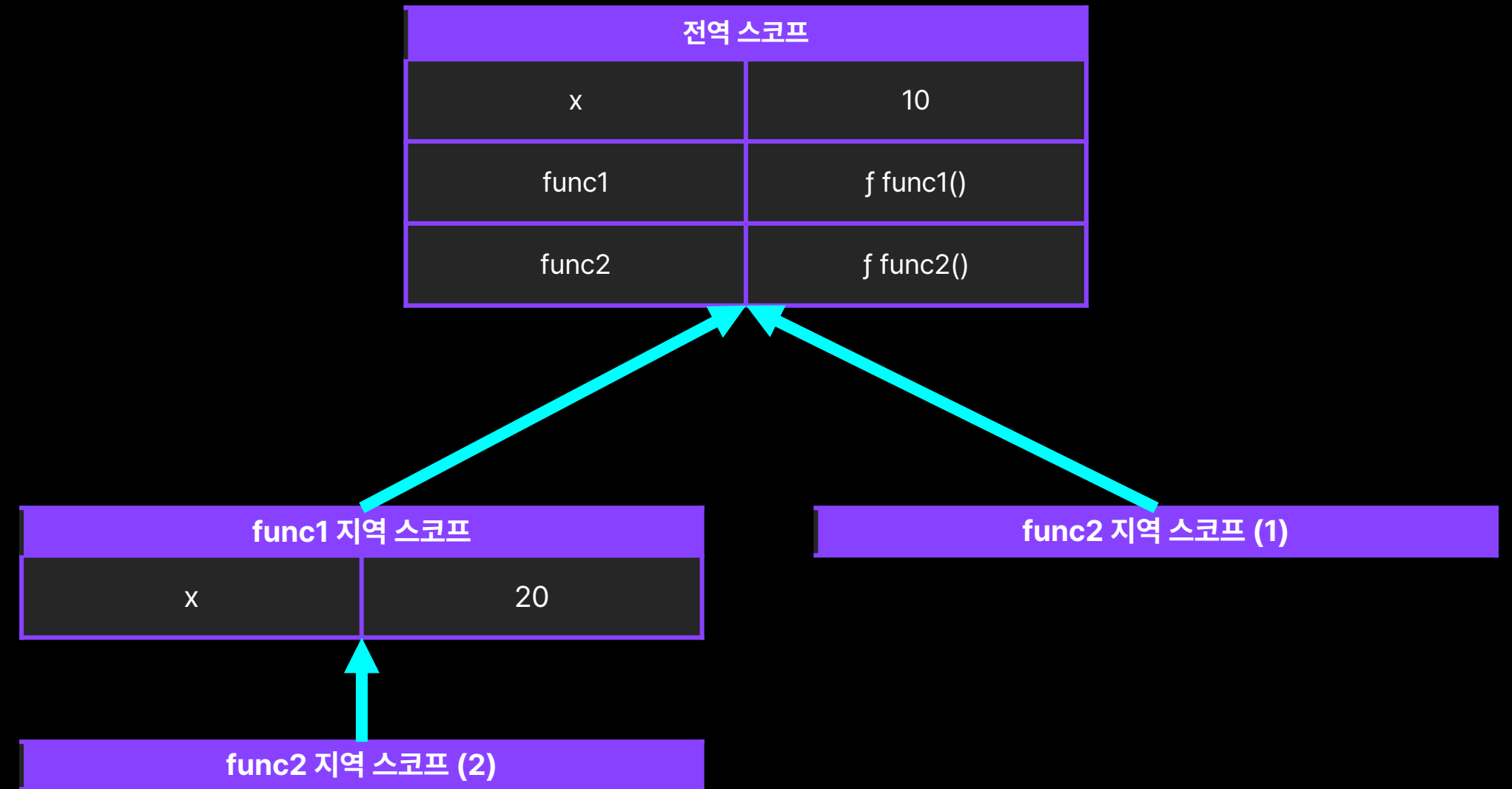
동적 스코프 (Dynamic Scope)

```
const x = 10;

function func1() {
  const x = 20;
  func2();
}

function func2() {
  console.log(x);
}

func1();
func2();
```



함수를 **호출**한 위치에서 상위 스코프를 결정하는 방식을 **동적 스코프**라고 한다.

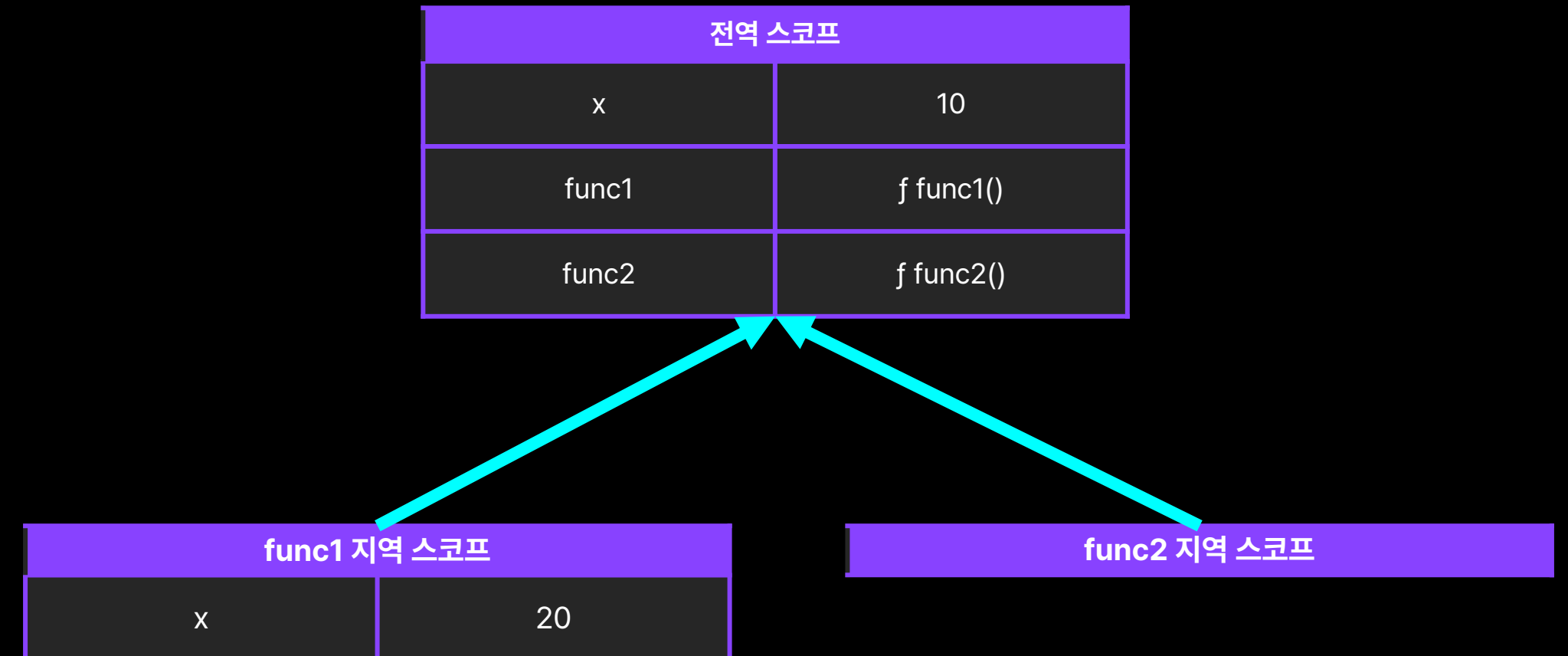
렉시컬 스코프 (Lexical Scope)

```
const x = 10;
```

```
function func1() {  
  const x = 20;  
  
  func2();  
}
```

```
function func2() {  
  console.log(x);  
}
```

```
func1();  
func2();
```



반면 함수를 정의한 위치에서 상위 스코프를 결정하는 방식을 렉시컬 스코프라고 한다.



렉시컬 스코프 (Lexical Scope)

```
const x = 10;
```

```
function func1() {  
  const x = 20;
```

```
  func2();  
}
```

```
function func2() {  
  console.log(x);  
}
```

```
func1();  
func2();
```



10 10

자바스크립트는 **렉시컬 스코프**를 따르므로,
왼쪽 코드의 실행 결과는 오른쪽과 같다.



1997

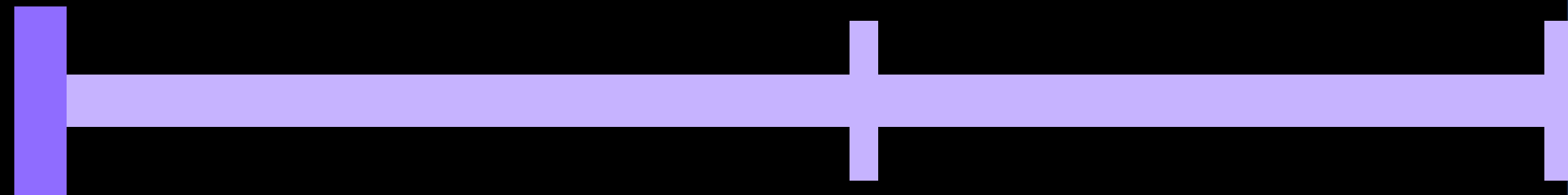
ECMAScript 1
(ES1)

1998

ECMAScript 2
(ES2)

1999

ECMAScript 3
(ES3)



자바스크립트는 1997년도부터
ECMAScript라는 사양으로 표준화 되어왔다.



09

cript 5
5)

2015

ECMAScript 2015
(ES6)

2016

ECMAScript 2016
(ES7)

2017

ECMAScript 2017
(ES8)

그리고 2015년에 업데이트 된 **ES6**(ECMAScript 2015)에서는
`let`, `const`를 비롯하여 중요한 큰 변화들이 많이 도입되었다.



2015

ECMAScript 2015 (ES6)

let, const

화살표 함수

(Arrow Function)

`() => { ... }`

옵셔널 체이닝

(Optional Chaining)

`user.contact?.email`

구조 분해 할당

(Destructuring Assignment)

`const { name, age } = hellobit;`

Spread 문법

(Spread Syntax)

`const combined = [...arr1, ...arr2];`

템플릿 리터럴

(Template Literal)

`const text = `Hello ${name}`;`

프로퍼티/메서드
축약

(Shorthand Properties/Method Names)

`{ name, age }`

배열 메서드

(Array Methods)

`forEach, map, filter, reduce` 등

⋮



화살표 함수

```
function hello() { ... } // 일반 함수
const hello = () => { ... }; // 화살표 함수

btn.addEventListener("click", function () { ... }); // 일반 함수
btn.addEventListener("click", () => { ... }); // 화살표 함수

function (a, b) { return a + b; }
(a, b) => a + b; // 중괄호 없이 작성하면 return 키워드를 생략하고 값을 반환할 수 있다.
```

화살표 함수는 ES6 이후에 도입된 새로운 함수 선언 형태로,
일반 함수보다 **간략하게 함수를 선언**할 수 있다.

옵션 체이닝 (Optional Chaining)

```
const user = {  
  profile: {  
    name: 'Alice'  
  },  
  numbers: [1, 2, 3]  
};  
  
console.log(user.profile.name); // 'Alice'  
console.log(user.contact.email); // error  
console.log(user.numbers[1]); // 2  
console.log(user.numbers[4]); // error
```

객체에서 존재하지 않는 프로퍼티에 접근하거나,
배열에서 범위를 벗어난 인덱스를 조회하려고 하면 에러가 발생한다.



옵셔널 체이닝 (Optional Chaining)

```
const user = {  
  profile: {  
    name: 'Alice'  
  },  
  numbers: [1, 2, 3]  
};  
  
console.log(user.profile?.name); // 'Alice'  
console.log(user.contact?.email); // undefined  
console.log(user.numbers?.[1]); // 2  
console.log(user.numbers?.[4]); // undefined
```

‘?.’ 연산자를 사용하면 존재하지 않는 프로퍼티/인덱스에 대해
에러 없이 undefined를 반환한다.

구조 분해 할당 (Destructuring Assignment)

```
const person = {  
  name: 'John',  
  age: 30,  
  city: 'New York'  
};  
  
const name = person.name;  
const age = person.age;  
const city = person.city;
```

ES6 이전에는 객체의 프로퍼티 값을 새로운 변수에 할당하려면,
각 변수를 선언할 때마다 프로퍼티를 하나씩 참조해야 했다.

구조 분해 할당 (Destructuring Assignment)

```
const person = {  
  name: 'John',  
  age: 30,  
  city: 'New York'  
};  
  
// 모든 프로퍼티를 할당할 필요는 없음  
const { name, age } = person;
```

ES6부터는 위와 같이 간단하게 객체의 프로퍼티 값을 변수에 할당할 수 있다.

구조 분해 할당 (Destructuring Assignment)

```
const person = {  
  name: 'John',  
  age: 30,  
  city: 'New York'  
};  
  
// name -> nameOfJohn  
const { name: nameOfJohn, age } = person;  
  
console.log(nameOfJohn); // 'John'  
console.log(age); // 30
```

각 변수의 이름을 다른 이름으로 변경할 수도 있다.

구조 분해 할당 (Destructuring Assignment)

```
const person = {
  name: 'John',
  age: 30,
  city: 'New York',
  contact: {
    email: 'john@elice.io',
    phone: '000-0000-0000'
  }
};

const { name, age, grade = 1, contact: { email } } = person;

console.log(name); // 'John'
console.log(age); // 30
console.log(grade); // 1
console.log(email); // 'john@elice.io'
```

기본값 설정, 객체 안의 객체 구조 분해 할당 또한 가능하다.

구조 분해 할당 (Destructuring Assignment)

```
const person = {
  name: 'John',
  age: 30,
  city: 'New York',
  contact: {
    email: 'john@elice.io',
    phone: '000-0000-0000'
  }
};

const { name, age, ...rest } = person;

console.log(name); // John
console.log(age); // 30
console.log(rest); // { city: 'New York', contact: { email: 'john@elice.io', phone: '000-0000-0000' } }
```

``...``을 이용해 할당되지 않은 나머지 프로퍼티는 객체 형태로 할당할 수 있으며,
이 객체는 보통 **rest** 라는 이름으로 설정한다.

→ Rest Syntax

구조 분해 할당 (Destructuring Assignment)

```
const person = {  
  name: 'John',  
  age: 30,  
  city: 'New York'  
};  
  
function introducePerson({ name, age }) {  
  console.log("Name is " + name);  
  console.log("Age is " + age);  
}  
  
introducePerson(person);  
// 'Name is John'  
// 'Age is 30'
```

함수의 인수로 전달되는 객체를 바로 구조 분해 할당할 수 있다.

구조 분해 할당 (Destructuring Assignment)

```
const numbers = [1, 2, 3, 4, 5];  
  
const [first, second] = numbers;  
  
console.log(first); // 1  
console.log(second); // 2
```

배열 또한 구조 분해 할당을 할 수 있다.

이때 변수의 이름은 원하는 대로 설정할 수 있다.

구조 분해 할당 (Destructuring Assignment)

```
const numbers = [1, 2, 3, 4, 5];  
  
const [first, second, ...rest] = numbers;  
  
console.log(first); // 1  
console.log(second); // 2  
console.log(rest); // [ 3, 4, 5 ]
```

배열에서도 `...`를 사용해 나머지 값을 배열 형태로 할당할 수 있다.

→ Rest Syntax



Spread 문법 (Spread Syntax)

```
...[1, 2, 3, 4, 5] -> 1, 2, 3  
..."Elice" -> 'E', 'l', 'i', 'c', 'e'  
...document.querySelector("li") -> <li></li>, <li></li>, <li></li>
```

이터러블*에 해당하는 여러 값의 집합을 펼쳐서
개별 값의 목록으로 만드는 문법이다.



Spread 문법 (Spread Syntax)

```
const arr1 = [1, 2, 3, 4, 5];  
const arr2 = [6, 7, 8, 9, 10];  
  
console.log([arr1, arr2]) // [ [ 1, 2, 3, 4, 5 ], [ 6, 7, 8, 9, 10 ] ]  
console.log(...arr1, ...arr2); // [ 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 ]
```

여러 배열을 결합할 때 간편하게 사용할 수 있다.



Spread 문법 (Spread Syntax)

```
const arr = [1, 2, 3, 4, 5];  
console.log(...arr); // 1 2 3 4 5  
  
function sum(a, b) { return a + b; }  
  
const arrToSum = [45, 23]  
sum(...arrToSum); // 68
```

또한 함수의 인수로 여러 값을 전달할 때 사용할 수 있다.



Spread 문법 (Spread Syntax)

```
const nameAndAge = { name: 'John', age: 30 }

const person = {
  ...nameAndAge,
  city: 'New York'
};

console.log(person);
// {
//   name: 'John',
//   age: 30,
//   city: 'New York'
// }
```

객체 또한 Spread 문법을 사용할 수 있다.
단, **객체 리터럴 내부에서만** 사용 가능하다.

구조 분해 할당 + Rest 문법 + Spread 문법

```
const person = {
  name: 'John',
  age: 30,
  city: 'New York'
};

function copyPersonWithNewName(person, newName) {
  const { name, ...rest } = person;
  return {
    name: newName,
    ...rest
  }
}

copyPersonWithNewName(person, "Hellobit"); // { name: 'Hellobit', age: 30, city: 'New York' }
```

Rest 문법

Spread 문법

구조 분해 할당과 Rest 문법, Spread 문법을 활용하면
특정 프로퍼티의 값만 바꾼 복사된 객체를 생성하는 로직을 구현할 수 있다.



템플릿 리터럴 (Template Literal)

```
const name = "Hellobit";  
const age = 20;  
  
// 기존  
console.log("My name is " + name + " and I am " + age + " years old.");  
  
// 템플릿 리터럴 사용  
console.log(`My name is ${name} and I am ${age} years old.`);
```

문자열 내부에 변수나 표현식을 포함해야 할 때,
``백틱 기호와 ``\$``를 이용하여 편리하게 문자열을 작성할 수 있다.



프로퍼티/메서드 축약 (Shorthand Properties/Method Names)

```
const name = "Hellobit";
const age = 20;

const hellobit1 = {
  name: name,
  age: age
}

const hellobit2 = {
  name,
  age
}

console.log(hellobit1); // { name: 'Hellobit', age: 20 }
console.log(hellobit2); // { name: 'Hellobit', age: 20 }
```

객체 리터럴에서 프로퍼티의 이름과 변수 이름이 동일할 때,
해당 프로퍼티 이름을 생략할 수 있다.



2015

ECMAScript 2015 (ES6)

let, const

화살표 함수

(Arrow Function)

`() => { ... }`

옵셔널 체이닝

(Optional Chaining)

`user.contact?.email`

구조 분해 할당

(Destructuring Assignment)

`const { name, age } = hellobit;`

Spread 문법

(Spread Syntax)

`const combined = [...arr1, ...arr2];`

템플릿 리터럴

(Template Literal)

`const text = `Hello ${name}`;`

프로퍼티/메서드
축약

(Shorthand Properties/Method Names)

`{ name, age }`

배열 메서드

(Array Methods)

`forEach, map, filter, reduce` 등

⋮



2015

ECMAScript 2015 (ES6)

let, const

화살표 함수

(Arrow Function)

() => { ... }

옵셔널 체이닝

(Optional Chaining)

user.contact?.email

구조 분해 할당

(Destructuring Assignment)

const { name, age } = hellobit;

Spread 문법

(Spread Syntax)

const combined = [...arr1, ...arr2];

템플릿 리터럴

(Template Literal)

const text = `Hello \${name}`;

프로퍼티/메서드
축약

(Shorthand Properties/Method Names)

{ name, age }

배열 메서드

(Array Methods)

forEach, map, filter, reduce 등

⋮

고차 함수 (Higher-Order Functions)

```
// 고차 함수: 함수를 인자로 받음
function repeat(n, action) {
  for (let i = 0; i < n; i++) {
    action(i);
  }
}

// 사용 예제
repeat(3, console.log); // 0 1 2
repeat(3, (index) => console.log(`Hello ${index}`)); // Hello 0, Hello 1, Hello 2
```

```
// 고차 함수: 함수를 반환함
function createMultiplier(multiplier) {
  return function(value) {
    return value * multiplier;
  };
}

// 사용 예제
const double = createMultiplier(2);
const triple = createMultiplier(3);

console.log(double(5)); // 10
console.log(triple(5)); // 15
```

고차 함수는 함수를 인자로 받거나, 함수를 반환하는 함수이다.



콜백 함수 (Callback Functions)

```
// 고차 함수: 함수를 인자로 받음
function repeat(n, action) {
  for (let i = 0; i < n; i++) {
    action(i);
  }
}

// 사용 예제
repeat(3, console.log); // 0 1 2
repeat(3, (index) => console.log(`Hello ${index}`)); // Hello 0, Hello 1, Hello 2
```

함수를 인자로 받는 고차 함수에서,
인자로 넘겨주는 함수를 **콜백 함수**라고 부른다.



콜백 함수 (Callback Functions)

```
var button = document.getElementById("btn");  
  
button.addEventListener("click", function () {  
    console.log("button clicked");  
});
```

‘addEventListener’에서 등록하는 이벤트 핸들러 함수 또한
콜백 함수의 일종이다.



콜백 함수 (Callback Functions)

forEach

map

filter

reduce

콜백 함수를 사용하는 또 다른 예시는
배열 메서드를 사용하는 것이다.

forEach

forEach

map

filter

reduce

```
const numbers = [1, 2, 3, 4, 5];

numbers.forEach((num, index) => {
  console.log(`Index ${index}: ${num}`);
});
```

```
// 출력:
// Index 0: 1
// Index 1: 2
// Index 2: 3
// Index 3: 4
// Index 4: 5
```

- `forEach`는 배열의 각 요소에 대해 콜백 함수를 **한 번씩 순서대로 실행**한다.
- 콜백 함수에는 각 차례의 배열 요소와 해당 요소의 인덱스 값이 인수로 전달된다.
- 반환 값은 없다.

forEach

map

filter

reduce

```
const numbers = [1, 2, 3, 4, 5];  
  
const doubled = numbers.map(num => num * 2);  
  
console.log(doubled); // [2, 4, 6, 8, 10]
```

- `map`은 배열의 각 요소에 대해 콜백 함수를 **실행한 반환 값을 모아 새로운 배열을 반환**한다.
- 콜백 함수에는 각 차례의 배열 요소와 해당 요소의 인덱스 값이 인수로 전달된다.
- **원래 배열의 값은 변경되지 않는다.**

forEach

map

filter

reduce

```
const numbers = [1, 2, 3, 4, 5];

const evenNumbers = numbers.filter(num => num %
2 === 0);

console.log(evenNumbers); // [2, 4]
```

- `filter`는 배열의 각 요소에 대해 **특정 조건을 만족하는 요소를 모아 새로운 배열을 반환**한다.
- 콜백 함수에는 각 차례의 배열 요소와 해당 요소의 인덱스 값이 인수로 전달된다.
- 콜백 함수는 조건의 결과를 **boolean 값으로 반환**해야 한다.
- **원래 배열의 값은 변경되지 않는다.**

forEach

map

filter

reduce

```
const numbers = [1, 2, 3, 4, 5];
```

```
const sum = numbers.reduce((accumulator, currentValue) => accumulator + currentValue, 0);
```

```
console.log(sum); // 15
```

초깃값

- `reduce`는 배열의 각 요소에 대해 콜백 함수를 호출하여 **누적된 결과를 반환**한다.
- 두 번째 인수로 초깃값을 지정할 수 있다.
- 콜백 함수에는 이전 단계까지의 누적 값과 현재 요소 값이 인수로 전달된다.
- 콜백 함수는 누적 값과 현재 요소 값으로 **계산한 결과를 반환**해야 한다.